COMPONENTES DE LÓGICA DE NEGOCIOS · PIA

# Sazón Urbano

API REST con lógica de negocio por capas: pedidos, cocina, inventario y promociones sincronizados

Equipo 3 · UANL FIME · Docente Miguel Ángel Gómez Marroquín

# El problema: pedidos, cocina, ventas e inventario no se sincronizan

### Hoy

Productos agotados que se siguen vendiendo, órdenes incompletas y cambios o cancelaciones que nadie puede rastrear. El inventario se revisa al final del día.

### Con la API

Un pedido no se confirma sin disponibilidad real. Al confirmar se descuenta el inventario en una sola transacción y cada cambio deja usuario, fecha y motivo.

**8 necesidades (N-01 a N-08)**

**15 reglas de negocio**

**15 historias · 5 casos de uso**

# Cuatro capas, dependencias hacia el dominio

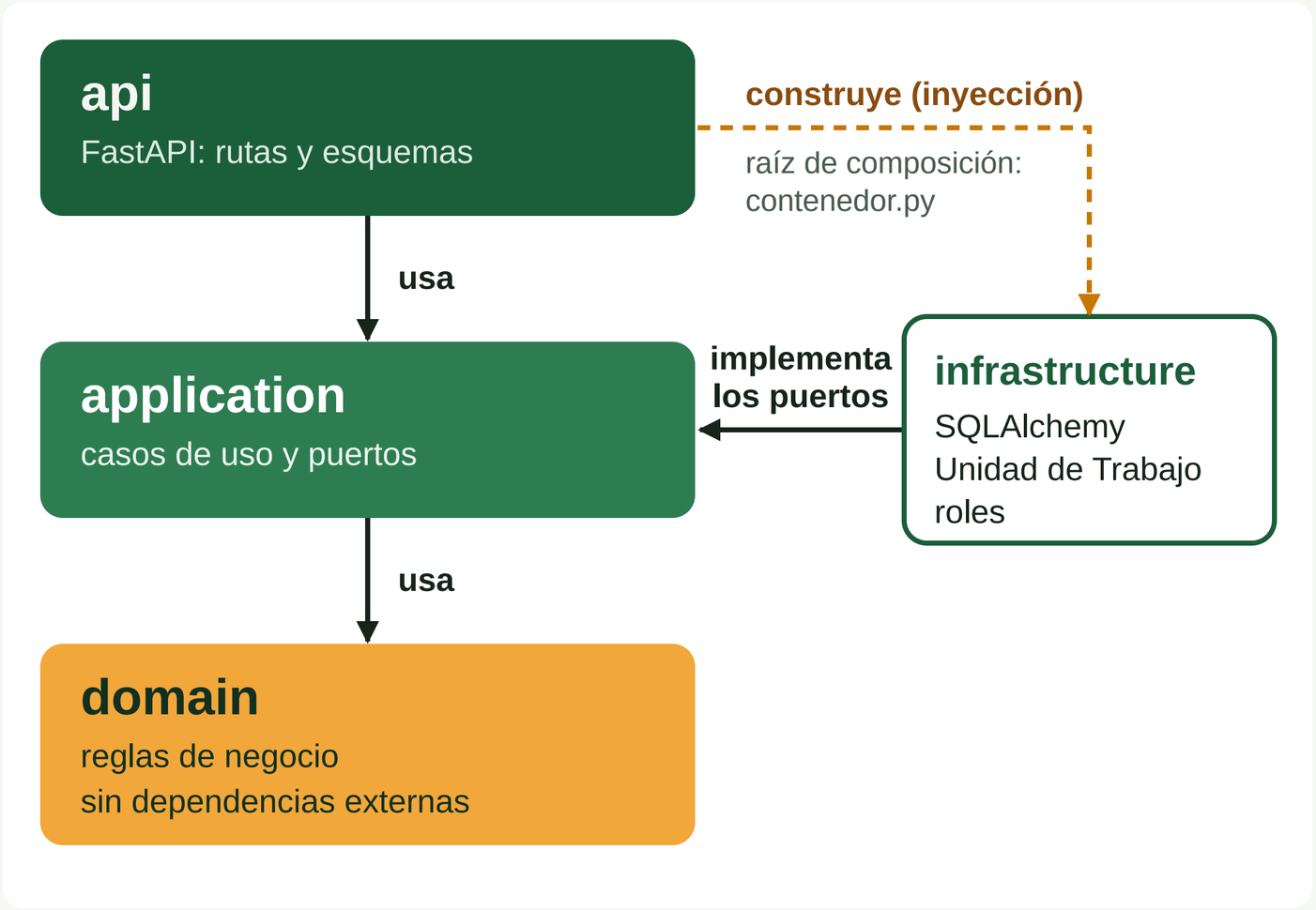
**api** traduce HTTP y errores.

**application** orquesta casos de uso y define puertos.

**domain** guarda las reglas, sin depender de nada.

**infrastructure** implementa los puertos: SQLAlchemy, Unidad de Trabajo y roles.

Resultado: el dominio se prueba sin base de datos ni HTTP.



# Las reglas viven en el dominio

### Aggregates

Pedido (líneas, historial, cancelación), Producto, Ingrediente y Promoción

### Specification

puede_procesarse (AND) y califica_beneficio (AND y NOT), componibles con & | ~

### Máquina de estados

6 estados y una matriz de modificar y cancelar

### Domain Events

15 eventos: bitácora en la transacción y publicación tras el commit

### Unit of Work

Confirmar descuenta inventario o revierte todo

### CQRS básico

Comandos con el dominio y consultas con DTO

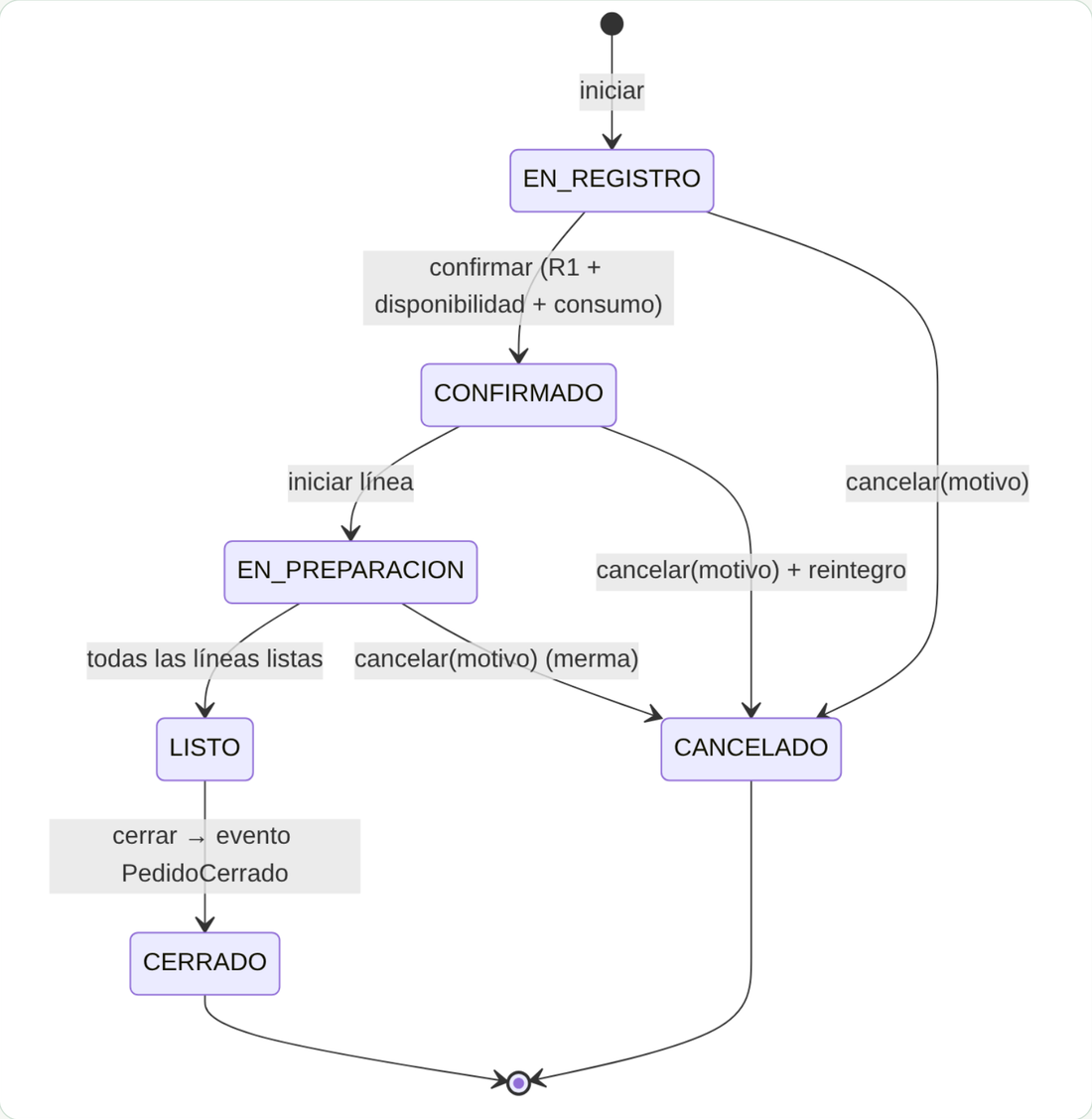
# Ciclo de vida del pedido

Modificar: en registro y confirmado.

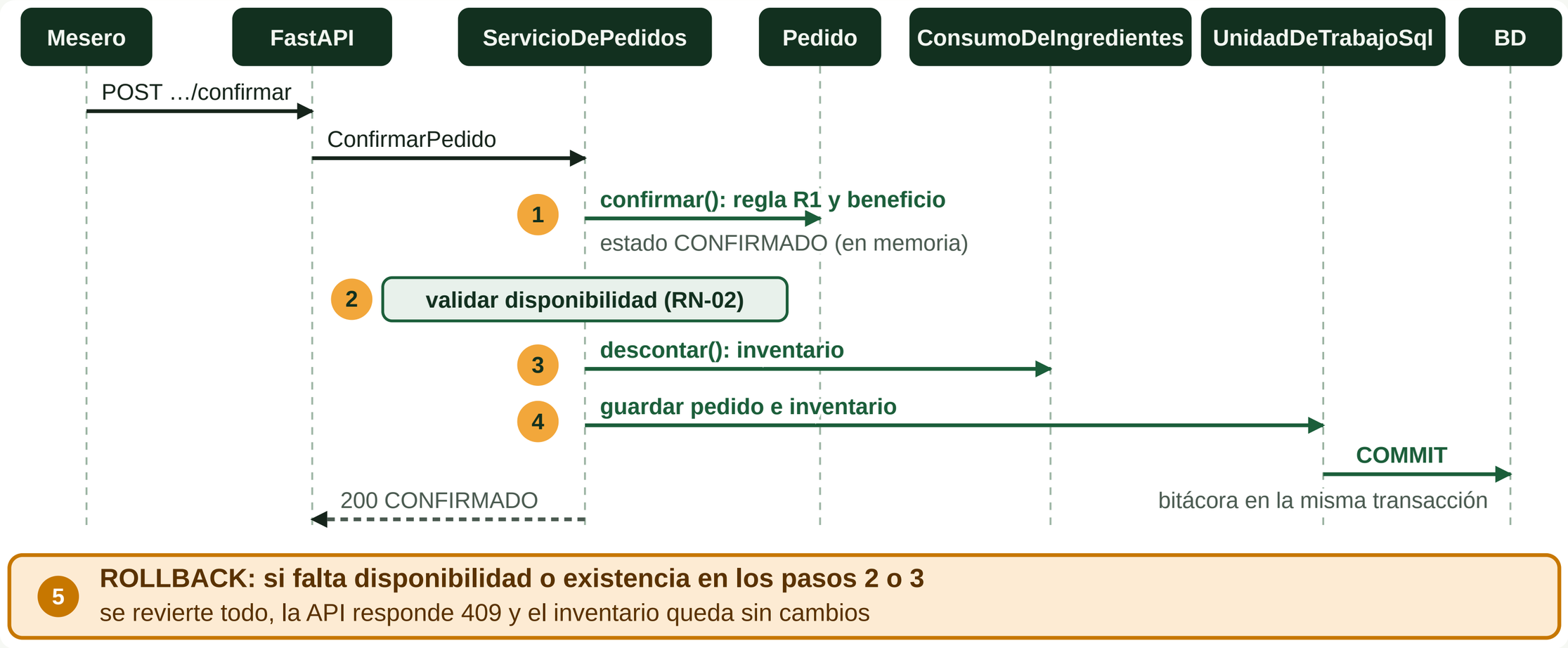
Cancelar: en registro, confirmado y en preparación, siempre con motivo.

Listo y cerrado ya no se cancelan: responde 409.

Cancelar un pedido confirmado reintegra los ingredientes; en preparación es merma.



# Confirmar un pedido es una sola transacción




Los eventos se publican después del commit. Probado con rollback: no quedan cambios parciales.

# El paquete del docente como prueba independiente

**120**

casos de prueba: 60 de R1 y 60 de R2

**20**

transiciones ejecutadas por la API

**0**

discrepancias contra el oráculo calculado desde los CSV

El LEEME pide documentar diferencias y no reemplazar los requisitos propios: estados, precios (se revisó SP-06) y reglas R1 y R2 están en la nota de adaptación.

# Calidad medida, no declarada

**274**
pruebas aprobadas; 1 omitida, 275 recopiladas

**98.51%**
cobertura de líneas y ramas; el mínimo es 80 %

**0**
observaciones de Ruff

**95** pruebas unitarias: las 95 aprobadas

**180** de integración: 179 aprobadas y 1 omitida

Las pruebas encontraron un defecto real, un choque de nombres al emitir eventos. Se corrigió y quedó cubierto con una prueba.

# Demo: la API en ejecución

Producto agotado: 409.

Confirmar: 200 y descuenta inventario.

Confirmar otra vez: 409.

Mesero cancela: 403. Encargado cancela: reintegra.

25 rutas en Swagger UI (/docs) y OpenAPI exportado.

```
$ curl -s localhost:8765/health
{"estado":"ok"}

$ POST /pedidos  {"canal":"MESA","cliente_id":"PED-C001"}   -> 201
pedido_id: PED-0F2307F9

$ POST /pedidos/PED-0F2307F9/lineas  {"producto_id":"PED-P011","cantidad":1}
{"codigo":"PRODUCTO_NO_DISPONIBLE","mensaje":"Pasta 11 está agotado"}
HTTP 409

$ POST .../lineas  {"producto_id":"PED-P001","cantidad":2}
HTTP 201
POST .../autorizar -> HTTP 200

$ POST .../confirmar
{
 "pedido_id": "PED-0F2307F9",
 "estado": "CONFIRMADO",
 "total": "580.00",
 "descuento": "0.00",
 "total_a_pagar": "580.00"
}
HTTP200

$ POST .../confirmar (segunda vez)
{"codigo":"TRANSICION_INVALIDA","mensaje":"El pedido no puede pasar de CONFIRMADO a CONFIRMADO."}
HTTP 409

$ POST .../cancelar con rol MESERO
{"codigo":"NO_AUTORIZADO","mensaje":"El rol MESERO no puede realizar CANCELAR_PEDIDO."}
HTTP 403

$ POST .../cancelar con rol ENCARGADO
CANCELADO {'motivo': 'cliente se fue', 'usuario': 'enc1', 'fecha': '2026-11-05T23:38:47'}

$ GET .../historial
2026-11-05T23:38:47 demo     Pedido iniciado | canal=MESA
2026-11-05T23:38:47 demo     Producto rechazado por disponibilidad | PED-P011: Pasta 11 está agotado
2026-11-05T23:38:47 demo     Línea agregada | PED-P001 x2
2026-11-05T23:38:47 demo     Pedido autorizado |
2026-11-05T23:38:47 demo     Pedido confirmado | enviado a cocina e inventario
2026-11-05T23:38:47 enc1     Pedido cancelado | motivo: cliente se fue

$ GET /inventario/ingredientes/ING-PED-P001 (tras confirmar y cancelar: reintegrado)
{'ingrediente_id': 'ING-PED-P001', 'nombre': 'Insumo base de Hamburguesa 01', 'existencia': '11.000', 'umbral': '3.000', 'es_critico': False}
```

# Trazabilidad de punta a punta

| Necesidad | Regla | Historia | Componente | Prueba |
|---|---|---|---|---|
| N-02 | RN-04 | US-06 | Pedido.confirmar | test_confirmar_exige… |
| N-03 | RN-02 | US-04 | DisponibilidadProductos | test_cu02_* |
| N-04 | RN-12 | US-06 | ConsumoDeIngredientes | test_rollback_* |
| N-05 | RN-07 | US-11 | Pedido.cancelar | test_cu04_* |

Cada fila une una necesidad del BRD con su regla, su historia, el código y la prueba que la verifica. Matriz completa en docs/architecture/trazabilidad.md y en la sección 10 del reporte.

# Docker, CI/CD y despliegue

### Docker Compose

db PostgreSQL con healthcheck, app y tests con perfil propio

### GitHub Actions

Ruff, pytest con mínimo 80 % en tres versiones de Python, PostgreSQL y trabajo Docker

### Render

render.yaml: servicio Docker, base gestionada y /health

12-Factor: toda la configuración va en variables de entorno y no hay secretos en el repositorio.

URL pública: [pegar la URL del despliegue con /docs]

# Conclusiones

Las reglas del negocio viven en un dominio probado, no repartidas en scripts.

Confirmar y cancelar mantienen pedido e inventario consistentes.

Cada regla se traza hasta una prueba automática.

Trabajo futuro: autenticación con JWT, seguimiento del repartidor, cola para actualizar ventas y medir conflictos de versión.